Document 17 — Interview Preparation (covers everything)

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

All earlier documents — this one pulls them together, it doesn't teach anything new.

How to Read & Practice This Document

- **What:** repeated interview practice on everything you've learned, at growing depth.
- **Why:** knowing something and being able to give a clear, well-sized answer about it, on the spot, under a bit of pressure, are two different skills — this document trains the second one specifically.
- **When:** keep doing this format as you finish each document, not just once at the very end.
- **How to practice:** for every question — answer first, out loud or in writing, before reading ahead or asking me anything. Then: **Question → your answer → my review → correction → the ideal answer**, given at four levels of depth.

Jump to: The Four-Depth Format · Question Coverage · Practice Exercises

The Story — what this document is actually building

By this point you've built real things — config loaders, agents, a RAG pipeline, a multi-agent system, tests, a deploy pipeline. But knowing how to build something and being able to explain it clearly, on the spot, to someone deciding whether to hire you, are two different skills — and only one of them gets tested in an interview. This document is entirely about training the second skill.

The format is the same every time: you answer a question first, cold, before anything gets corrected for you — because a real interview never hands you the answer key first either. Then you get the same answer stretched across four depths: a 30-second version, a normal version, a deep technical version, and a senior version that talks about trade-offs and failure modes. Learning to give the *right-sized* answer — not the longest one you know, the one that actually fits the question being asked — is most of what separates a strong interview from an over-explained one.

The three practice exercises below build on each other. First, cold recall — can you say back a fact from a document you finished weeks ago, with no notes. Then, the follow-up template — can you survive the "why not X, how would you scale it" questions that come *after* your first answer, which is where most interviews are actually won or lost. And finally, the hardest one: explaining your own project's design to someone who has never seen it, which is exactly what you'll be asked to do in nearly every real interview you'll ever have for this kind of role.

The Four-Depth Format (used for every question)

1. **30-second answer** — what you'd say if someone interrupted you right away.
2. **Normal answer** — what you'd say in an ordinary interview response.
3. **Deep, technical answer** — what you'd say if pushed on exactly how it works.
4. **Senior-level answer** — what you'd say including trade-offs, failure types, and a real (or realistic) production example.

Go Deeper (Optional)

No new material — this document pulls its questions straight from every "Interview Topics Preview" section across Docs 01-16. Look back at those before a session here.

Question Coverage (per document you've already finished)

- 5 basic + 5 intermediate + 5 senior-level + 3 scenario-based + 3 debugging + 2 system-design questions, taken from that document's topics.
- A standing follow-up you should expect for any answer: *Why? Why not X? How? What happens underneath? What happens if it fails? How would you scale it? How would you cut its cost? How would you debug it?*

Below is a worked example of that formula for 3 documents spanning the curriculum — one early, one middle, one late — pulled from each document's real Core Concepts and Interview Topics Preview. Use these as-is for live practice, or as a model for building the same kind of list from any other finished document. No hints or solutions here on purpose — answer these live with the mentor, the same way this whole document runs.

Document 01 — Python Foundations

Basic

- What does a virtual environment do, and why does every project need its own?
- What's the difference between an error and just returning a special "something went wrong" value?

- Why should secrets live in a `.env` file instead of directly in code?
- What does `logging` give you that `print()` doesn't?
- What does a type hint like `-> Config` actually tell a reader of the code?

Intermediate

- How does `python -m venv .venv` plus `requirements.txt` keep two projects' dependencies from colliding, and why doesn't `.venv` itself get committed to git?
- Why is catching the broad `Exception` class usually the wrong move, and how does the error family tree let you catch something more specific instead?
- How does a good config loader avoid a confusing "401 Unauthorized" three functions deep, and what should it do differently at startup?
- How do logging handlers let the same log line go quietly to a file in production and loudly to the console locally, without changing the log call itself?
- Does a wrong type hint crash your program? What does it actually do instead?

Senior-level

- What's the real danger of a mutable default argument like `def f(items=[])`, and why does it only surface as a bug sometimes?
- If a required `.env` key is silently missing, what's the worst place for that to fail — and how do you make it fail at startup instead?
- What breaks in production if every log message gets written at `ERROR` level regardless of how serious it actually is?
- What's the cost of an `except: pass` block, and how would you go about finding a bug it's been silently swallowing?
- How would you migrate a codebase full of `print()` calls to real logging without editing every call site by hand?

Scenario-based

- A teammate installs your project and immediately hits an import error from a package version conflict with their other project. What do you tell them?
- Your service crashes in production with "401 Unauthorized" from an external API, but the `.env` file on that machine really does have the key set. Where do you look first?
- A teammate writes `def call(retries=[])` for a function that collects retry attempts across calls. What's wrong with it, and what do you tell them to write instead?

System-design

- Design a config loader for a service with 6 required environment variables and 3 optional ones. What does it check at startup, and what does a failure message look like?
- Design a logging setup for a project with 3 modules that each need their own log file, but all warnings and errors also need to land in one combined log. Which Doc01 pieces do you use,

and how do they fit together?

Document 08 — RAG

Basic

- What is an embedding, in plain terms?
- Why do documents get split into chunks before they're embedded?
- What does "top-k" mean in a vector search?
- Does RAG stop a model from making things up?
- What's the difference between a retrieval failure and a generation failure?

Intermediate

- Why do the question and the documents need to be embedded with the same embedding model?
- Why is choosing a chunk size a real design decision, not just a technical detail?
- What is the "lost in the middle" problem, and why does it mean more retrieved context isn't automatically better?
- How does hybrid search combine keyword and vector search, and why doesn't either one alone work well for something like a product SKU?
- What does a reranker do that the first-pass vector search can't, and why not just run the reranker over everything?

Senior-level

- Your RAG system scores well on retrieval metrics but users still report wrong answers. What does that tell you about where to look next?
- How would you decide whether to raise `k`, add metadata filtering, or add a reranker, if answers are coming back inconsistent?
- What's the security risk of skipping metadata filtering in a multi-tenant RAG system, and how is that different from a plain relevance problem?
- How would you cut the cost of a pipeline that reranks every single query, without dropping the reranker entirely?
- How would you debug a case where the chunk with the exact answer never shows up in the top-k results at all?

Scenario-based

- A user asks about "the 2024 policy" and the system returns a similarly-worded chunk from the 2019 policy instead. What do you check first, and what do you add?
- Your evaluation shows strong retrieval metrics but weak generation metrics on the same test set. What does that tell you to fix, and what would you NOT waste time on?

- A support tool starts returning documents that belong to a different customer than the one asking. What went wrong, and what's the fix?

System-design

- Design a RAG pipeline for a multi-tenant support tool where each customer's documents must stay isolated, and exact product codes need to be found reliably. Which Doc08 pieces do you include, and why?
- Design an evaluation process for a RAG system, run before it ships, that can tell you specifically whether a wrong answer is a retrieval problem or a generation problem.

Document 11 — Multi-Agent Systems

Basic

- What's the first question to ask before splitting a task across multiple agents?
- What does the supervisor pattern do, in plain terms?
- What's the difference between shared state and an agent's own private scratchpad?
- Why isn't a 4-agent pipeline "roughly the same speed" as one single agent?
- What does the `Command` object let a node do, in one step?

Intermediate

- How does a sequential pattern differ from a supervisor pattern, and when does each one actually fit better?
- Why can putting too much into shared state hurt a downstream agent, even though more information sounds like it should help?
- What are the three real options when one agent in a pipeline fails or returns something unusable, and when would you pick each one?
- Why is the older "text code" routing pattern (returning something like `"ROUTE_TO_WRITER"`) more fragile than using `Command`?
- What's the trade-off of a hierarchical (supervisors-of-supervisors) pattern versus a single supervisor?

Senior-level

- How would you defend using 4 agents instead of 1 well-prompted agent, if an interviewer pushes back that it sounds like over-engineering?
- What's a "state-leaking" failure in a multi-agent system, and how would you catch it before it reaches a user?
- How would you cut the cost of a supervisor-routed pipeline without collapsing it back down to a single agent?
- What's the risk of a Generator → Critic → Revision loop with no hard limit on revisions, and how do you bound it?

- When would a peer-to-peer or blackboard pattern actually beat a supervisor, and what do you give up by not having a central router?

Scenario-based

- Your supervisor occasionally routes work to the wrong specialist agent, and nobody notices until the final output looks off. What would you add to catch it earlier?
- Two parallel agents research the same topic independently and come back with conflicting facts. What's your plan for merging their results?
- A teammate wants to add a 5th agent "just in case it's useful later" to a working 4-agent pipeline. What do you ask them before agreeing?

System-design

- Design a multi-agent system for a task with three genuinely ordered phases, one point where results might need to loop back to an earlier phase, and a hard limit on how many times that loop can run. Which pattern(s) do you combine, and why?
- Design the shared state object for a 3-agent research-and-write pipeline — decide exactly what goes in shared state versus stays private to one agent, and justify each field.

Practice Exercises

Why only 3 here, not 5: this document's real practice is live, back-and-forth interview simulation — these 3 give you a fixed starting point; the actual depth comes from the live evaluation and correction after each one.

Where your answers live: `17_interview_preparation/practice/` (`mkdir -p practice`). Write each answer down before comparing — these are spoken answers, so each one is a short Markdown file:

```
practice/
├── cold_recall.md           Basic
├── followup_template.md     Intermediate
└── project_walkthrough.md   Real-world
```

Why each file exists:

- `cold_recall.md` — your cold, no-notes answers, so you can see what you actually retained, not just recognised.
- `followup_template.md` — your first answer *and* your answers to the standing follow-ups, written before the interviewer asks them.
- `project_walkthrough.md` — your four beats (problem, shape, one decision, one limitation) for one project, reusable for any other.

Jump to an exercise: Basic · Intermediate · Real-world

Basic — cold recall

- **What:** answer 5 basic questions from a document you've finished, cold, with no notes.
- **Why:** this checks whether you actually retained the material, or only recognized it while reading — a real interview gives you no notes either.
- **When you'll hit this for real:** the first 10 minutes of almost any technical interview — basic recall questions, meant to confirm you know the fundamentals before going deeper.
- **How to do it:** pick a finished document, close it, and answer 5 of its "Interview Topics Preview" items out loud or in writing, timing yourself.
- **Save as:** `practice/cold_recall.md`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — apply the follow-up template yourself

- **What:** answer 3 scenario-based questions, then apply the standing follow-up questions to your own answer, before I do.
- **Why:** the actual senior skill isn't answering the first question well — it's anticipating the follow-up before it's asked, which is what this exercise trains directly.
- **When you'll hit this for real:** any interview past the junior level — a good interviewer always pushes past your first answer.
- **How to do it:** answer a scenario question, then immediately ask yourself "why not X, how would I scale it, how would I debug it" and answer those too, before checking against the ideal answer.
- **Save as:** `practice/followup_template.md`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — a cold interview on your own project

- **What:** pick one of your own projects and let me interview you about its design, as if I've never seen it before.
- **Why:** explaining your *own* design under real questioning is a different skill than reciting facts — this is the closest simulation to an actual interview this curriculum can give you.
- **When you'll hit this for real:** any interview where you're asked to walk through a project from your portfolio — this happens in nearly every real AI engineering interview.
- **How to do it:** say "interview me about Project 4" (or any project) and answer as if the interviewer has never seen your code — don't assume shared context.
- **Save as:** `practice/project_walkthrough.md` (your four beats, written before the live session).
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Move On When

You can answer a cold, scenario-based question about Project 4's design at a senior level, with no notes. Full details: CURRICULUM.md §4.

This document runs live. Say **START DOCUMENT 17** (you can name which document's topics to focus on) when ready.